COMPLETE PROJECT EXPLANATION

Lazarus Mesh

An autonomous recovery marketplace that bargains, scopes payment authority, purchases verified access, reconstructs data, and releases rewards only after proof.

Pay for verified recovery, not promises.

01	02	03
Rain	Monad	End-to-end
Cards for agents	Agentic commerce	Merchant marketplace

Hackathon MVP - local-first, remotely integrated, sandbox/testnet aware, and explicit about every trust boundary.

Repository snapshot: preview/x402-live at b41f3a6

Prepared 9 August 2026

SECTION 01

Executive summary

Lazarus Mesh turns a missing-but-identifiable digital artifact into a bounded autonomous commerce mission: discover, bargain, authorize, retrieve, verify, and complete.

3	$9.75	8/8	186
challenge tracks	**accepted quote**	**remote pieces**	**tests passing**
One integrated submission	Down from a $12.00 ask	Pinned and reconstructed	Full suite on this snapshot

The one-sentence product

A policy-bound buyer agent negotiates with an independent recovery merchant, creates narrowly scoped payment authority, downloads a sponsor-pinned artifact, verifies every byte, and advances a proof-conditioned bounty lifecycle.

What is genuinely external

Authenticated remote merchant bargaining and provider downloads; Ed25519 signature verification; Rain sandbox card calls when enabled; optional Monad test-USDC x402 settlement when separately armed.

What remains modeled

The full Monad bounty registry, independent verifier network, reseeding, persistent replicas, and production-money merchant settlement are not yet live.

Why it matters

The demo proves that agent autonomy can be useful without giving an agent unrestricted wallet authority or trusting a merchant's promise about data quality.

Verified repository snapshot

Evidence	Current result	Interpretation
Remote acceptance	Two rounds; signed $9.75 quote; 8/8 pieces; final root match	Real API and byte-verification path; $0.00 charged in that run
Rain sandbox	$9.75 at MCC 5734 settled; unrelated $9.00 attempt declined	Real sandbox API behavior; no production funds move
Monad x402	Capped buyer/seller path and protected-preview gates implemented	Can move test USDC when correctly armed; full bounty is still local
Automated tests	186 passed, 0 failed	Uses fakes for external rails; tests do not claim a live transaction

Operational snapshot - 9 August 2026

The public Vercel Production build is deployed, and Rain plus remote negotiation health are passing, but its remote provider health currently returns 503 PROVIDER_REQUEST_FAILED; do not present the public recovery path as healthy today. The protected Preview is access-controlled and readiness-healthy for Rain sandbox and Monad x402, but no x402 payment has been executed from that Preview yet. Its active signer is the dedicated raw-key fallback, while the Privy path remains implemented and test-covered.

SECTION 02

The problem and product thesis

Content-addressed data can remain identifiable after its last available copy disappears. Conventional checkout pays for access, not for verified recovery.

The failure

A valid content commitment exists, but availability is zero and no complete source can be reached.

The market gap

Merchants and providers can quote for archival egress, but buyers need price discipline, legal scope, payment limits, and proof of correct delivery.

The Lazarus answer

Treat recovery as an auditable mission whose payment and reward authority is derived from a signed quote and released only as verification milestones pass.

Default demonstration mission

Restore the CC0 Rainfall Dataset

The bundled mission begins with a known SHA-256 commitment, explicit CC0 rights evidence, zero modeled seeders, a $20.00 USD budget reference, and a $5.00 provider bounty reference. Local mode uses a 24-piece fixture. Remote mode uses the merchant's sponsor-pinned eight-piece artifact.

Design principles

- Pay for evidence. A merchant quote must be signed and policy-valid; recovered bytes must match the pinned manifest and full-content commitment.
- Bound autonomy. The orchestrator proposes counters and payment intents, while deterministic policies decide whether authority may be created or used.
- Separate values. Mission accounting, merchant spend, provider bounty, test USDC settlement, Rain sandbox rUSD collateral, and MON gas are not conflated.
- Truthful interfaces. The dashboard labels local simulation, sandbox operations, testnet execution, and external merchant behavior separately.
- Authorized content only. Missions require rights attestation and are intended for public-domain, openly licensed, creator-authorized, or enterprise-owned data.

Out of scope by design

Not this	Because
A torrent or piracy search engine	The system is restricted to legally authorized material with explicit rights evidence.
An unrestricted AI shopper	There is no free-form wallet authority; merchant, MCC, purpose, amount, count, expiry, and rail are bounded.
A production bank/card product	Rain usage is sandbox-only and the public demo does not move real money.
A completed decentralized storage network	Replica creation, verifier independence, and long-term reseeding remain modeled.

SECTION 03

One product across all three challenges

The submission is designed as a single commercial loop, not three disconnected sponsor demos.

Challenge 1 - Rain

The accepted quote becomes scoped card authority. Lazarus sends the amount, MCC allowlist, and expiry to Rain's sandbox, while application policy additionally binds merchant identity, purpose, content root, one-time use, and exact quote terms.

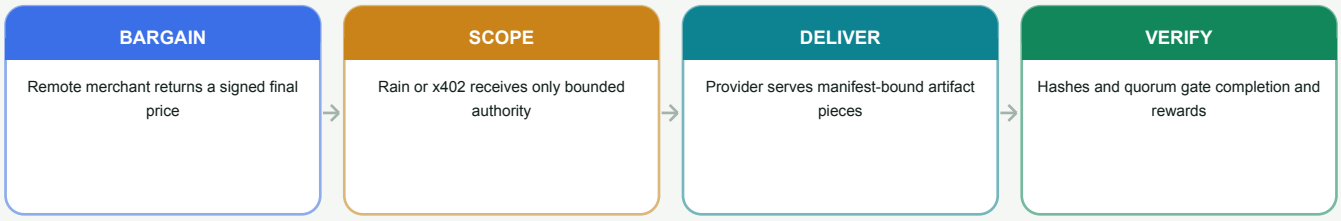
Challenge 2 - Monad

The protected x402 path prices an availability resource in official test USDC on Monad testnet. A restricted payer signs EIP-3009 authorization, the seller/facilitator settles, and the buyer independently verifies the exact confirmed Transfer.

Challenge 3 - End-to-end commerce

An independent merchant receives structured offers, counters, signs a binding quote, serves artifact pieces, and exposes an operator console. Lazarus bargains, purchases, reconstructs, verifies, and completes the mission.

How the tracks reinforce one another



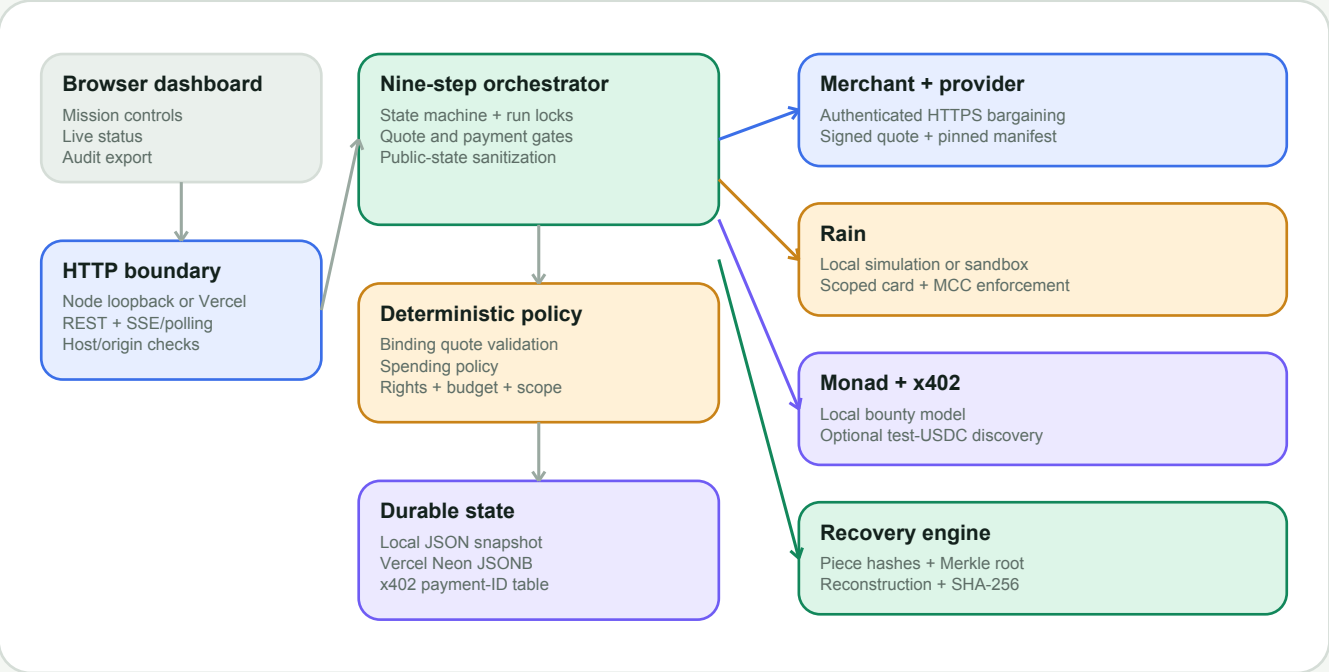
Judge-facing claim

Lazarus Mesh demonstrates real agent autonomy with guardrails: the agent makes progress independently, but it cannot widen its own budget, change the recipient, accept unsupported legal terms, reuse a quote, or mark recovery complete without cryptographic evidence.

SECTION 04

System architecture

A dependency-light browser and Node runtime coordinate deterministic policy with swappable merchant, payment, chain, recovery, and persistence adapters.



Layer	Responsibility	Trust boundary
Browser	Mission creation, step/run controls, metrics, transcript, receipts, audit export	Never receives server secrets, PAN/CVC, raw provider bytes, or private signing material
HTTP runtime	Node loopback server or Vercel function; REST; SSE locally and polling on Vercel	Validates host/origin, body limits, paths, and public serialization
Orchestrator	Nine ordered actions, run locks, idempotency, checkpointing, lifecycle transitions	Cannot bypass quote and payment policies
Adapters	Local/remote negotiation, Rain, Monad/x402, recovery, persistence	Every external response is bounded and validated before use
Durable state	Local audit snapshot or Neon JSONB plus x402 settlement records	Runtime binding prevents one mode from adopting another mode's authority

Technology profile

Frontend

Semantic HTML, responsive CSS, and dependency-free browser JavaScript.

Backend

Node.js 20+, CommonJS, native HTTP plus focused packages for Neon, viem, and x402.

Deployment

Vercel serverless API with Neon Postgres state; local server remains loopback-only.

SECTION 05

Actors, wallets, and trust

Rain and Monad are rails. The merchant and provider are counterparties. The buyer policy remains the authority over spend.

Actor	What it does	What it does not do
Mission principal	Attests rights and defines budget, currency, and deadline	Does not hand a blank wallet to the agent
Lazarus buyer policy	Runs the fixed workflow, counters, and submits exact payment intents	Is not an LLM with free-form purchase authority
Lazarus Recovery Merchant	Receives offers, counters deterministically, signs binding quotes	Does not control Lazarus policy or verification
Artifact provider	Returns the pinned manifest and ordered pieces	Cannot redefine the trust root it is being checked against
Rain	Provides scoped sandbox card and transaction APIs	Is not a merchant, recovery provider, or verifier
Monad/x402	Provides optional test-USDC payment evidence for paid discovery	Does not make the local bounty contract live
Payer wallet	Signs one tightly bounded authorization	Does not bargain or receive merchant credentials
Payee wallet	Receives test USDC; only its public address is needed	Needs no private key in Lazarus to receive
North/East/West verifiers	Provide scripted quorum state in the demo	Are not independent external verifier services

Communication map

Buyer <-> merchant

Authenticated HTTPS: offers, counters, quote, key ID, signature, session state.

Buyer <-> provider

Authenticated HTTPS: live manifest and bounded artifact-piece downloads.

Buyer <-> Rain

Server-only sandbox API key: collateral simulation, card, authorize, settle, reverse, reconcile.

Buyer <-> x402 seller

Protected same-origin transport in Preview: 402 requirement, signed payment, durable payment ID, paid result.

Browser <-> Lazarus

Same-origin mission controls and sanitized state. No server credentials cross this boundary.

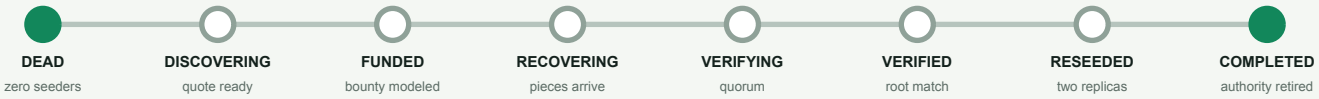
Merchant console

Shows settings, sessions, counters, signed deals, and audit events; proposed bounty is not yet persisted/displayed.

SECTION 06

The complete nine-step mission - part 1

The state machine moves forward only. Discovery and bargaining are separate from payment; payment is separate from proof of recovery.



Step 1 - Discover and bargain

The x402 adapter obtains availability intelligence for the content root. Then a separate merchant session runs the bounded $12.00 -> $9.00 -> $10.50 -> $9.75 transcript. Remote mode verifies the final Ed25519 quote.

Step 2 - Fund the recovery bounty

The local Monad-style ledger creates a mission/content-root bounty using the selected-currency equivalent of the $5.00 reward and $2.00 provider collateral reference. No onchain escrow is funded.

Step 3 - Claim, scope, challenge, settle

Lazarus revalidates the quote, records the provider claim, creates narrow Rain authority, sends a deliberately unrelated $9.00 challenge, and settles only the exact accepted archive purchase.

Step 3 is the key commercial gate

Before authority exists	Authority created	Proof recorded
Quote signature and digest match; session, root, currency, terms, amount, expiry, budget, and approval threshold all pass	One mission; one merchant; MCC 5734; archival_egress purpose; exact amount; one transaction; short expiry	Blocked attempt stays declined; approved purchase records a safe receipt; quote becomes consumed

Reference spend after step 3

USD mission policy usage is exactly $9.76: $0.01 discovery plus $9.75 archive allocation. The $5.00 bounty and $2.00 collateral reference are tracked separately and do not inflate mission spend.

SECTION 07

The complete nine-step mission - part 2

Recovery progresses in batches, but rewards and completion advance only after every configured piece and the reconstructed artifact have been verified.

Step	Action	Evidence and state
4	Recover first batch	Reads the first dynamic batch from the local fixture or authenticated remote provider; each visible piece reports progress and a hash prefix.
5	Recover second batch	Advances through the second one-third milestone; the mission remains RECOVERING.
6	Recover and verify every piece	All 24 local or 8 remote pieces must match SHA-256. North and East pass; quorum becomes 2/2; state enters VERIFYING.
7	Reconstruct and release to 70%	Concatenates ordered pieces; full SHA-256 and Merkle root must match; two unique attestations are recorded; USD reference release reaches $3.50.
8	Model retained availability; release to 90%	Records two simulated seeders and releases another $1.00 reference; state becomes RESEEDED.
9	Release remainder and retire authority	Releases the final $0.50 reference, completes the provider, disables local card authority, and finalizes reconciliation events.

24 of 24 pieces verified against the manifest

Execution locks

Only one run or manual step may mutate a mission. Concurrent execution returns a conflict.

Reset generation

A reset increments an internal generation so an older asynchronous run cannot overwrite the new state.

Event bounds

The mission keeps at most 100 newest-first audit events and exposes a sanitized JSON export.

Important modeling boundary

The two seeders, verifier independence, and full bounty receipts are demo-state transitions. The software does not start BitTorrent/IPFS nodes, contact an external verifier network, or submit bounty lifecycle transactions to Monad.

SECTION 08

Merchant bargaining and signed quotes

The merchant participates through a structured protocol. Bargaining is deterministic, bounded, replay-aware, and independent from the payment rail.

Sequence	Speaker	Action	Amount	Meaning
Offer	Merchant	Initial ask	$12.00	Configured ceiling and starting price
Round 1	Buyer	Counter	$9.00	Mission target price
Round 1	Merchant	Counter	$10.50	Deterministic midpoint bounded by the merchant floor
Round 2	Buyer	Counter	$9.75	Best policy-compliant midpoint
Round 2	Merchant	Accept + sign	$9.75	Floor met; 15-minute binding quote issued

$2.25
negotiated savings
From the initial ask

18.75%
discount
Deterministic reference

2
buyer counter rounds
Policy allows up to three

15 min
quote lifetime
Rechecked before spend

What the quote commits to

The canonical SHA-256 digest covers quote ID, session ID, merchant and provider identities, MCC, content root, purpose, amount, currency, exact legal terms, binding status, issue time, and expiry. Remote mode then verifies the digest with the pinned merchant Ed25519 key.

Exact accepted terms

Term	Accepted value
Purchase model	One-time
Service	Archival egress
Auto-renewal	False
Data sharing	False
Exclusivity	False

Fail-closed quote checks

Identity

Session, merchant, provider, MCC, content root, and purpose must match.

Economics

Currency, positive amount, ceiling, total budget, and approval threshold must pass.

Integrity

Digest, Ed25519 signature, key fingerprint, issue time, expiry, terms, and round count must pass.

SECTION 09

Economics, budgets, and multiple currencies

The project supports five mission-accounting currencies, while each external rail retains its own settlement asset and operational boundary.

Reference item	USD amount	Mission spent?	Notes
Availability discovery	$0.01	Yes	Local x402-shaped receipt or optional test-USDC payment
Merchant initial ask	$12.00	No	An offer, not a purchase
Accepted archive quote	$9.75	Yes	Only after quote and spending policy pass
Unrelated challenge	$9.00	No	Deliberately denied
Recovery bounty	$5.00	Separate	Local reward/escrow model
Provider collateral	$2.00	Separate	Local Monad-style ledger
Mission budget	$20.00	Ceiling	External-style spend ceiling

Mission accounting catalog

Currency	Minor units per reference USD	Where it works
USD	100 cents	Local; current remote merchant; Rain sandbox mission
EUR	92 cents	Local accounting only in current release
GBP	78 pence	Local accounting only in current release
CAD	137 cents	Local accounting only in current release
AUD	152 cents	Local accounting only in current release

Mission currency

USD/EUR/GBP/CAD/AUD control display, budgets, bargaining, savings, and local reward values.

Rain settlement layer

USD mission accounting and sandbox rUSD collateral. No current EUR/GBP/CAD/AUD Rain calls.

Monad settlement layer

Official six-decimal test USDC for x402; MON is used by the settlement submitter for gas.

Do not confuse reserve with account balance

In the default demo, $2.25 can appear both as negotiated savings ($12.00 minus $9.75) and as demo service reserve remaining after $9.76 of simulated policy usage. Neither is a live Rain wallet balance, and neither proves a failed or successful real-money payment.

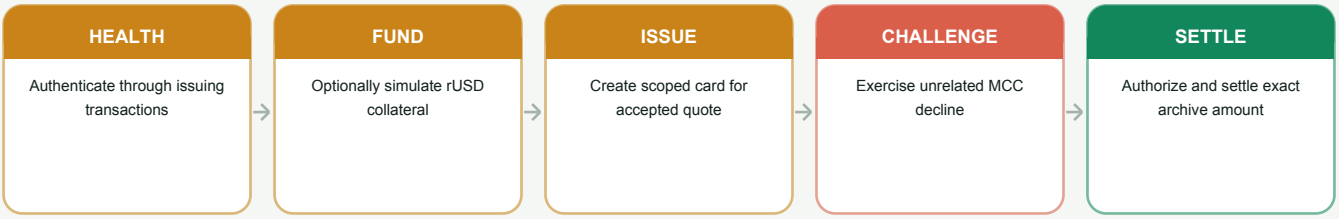
Rate model

The fixed lazarus-demo-reference-v1 table is deterministic and uses conservative integer rounding. It is not a market FX feed, rate-locking service, treasury system, or production multi-currency card product.

SECTION 10

Rain: scoped cards for agents

Rain provides the conventional-commerce control rail. Lazarus adds the quote, task, content, and mission policy that the sandbox card endpoint does not natively express.



Control	Lazarus policy	Rain sandbox request
Exact $9.75 quote	Required	Amount ceiling sent
Merchant identity	Required	Not represented in card creation
MCC 5734	Required	Required
Quote expiry	Required	Required
One transaction	Required	Not represented in card creation
Purpose/content root	Required	Not represented in card creation
Quote digest/terms	Required	Not represented

Verified sandbox evidence

Approved and blocked paths were both exercised

The repository records an end-to-end Rain sandbox run in which a real sandbox scoped card settled the $9.75 archive authorization at MCC 5734, while the deliberate $9.00 unrelated-merchant/MCC challenge was declined and visible in Rain's sandbox transaction ledger.

This is a sandbox demonstration. No production card purchase or real merchant funds moved.

Safety and reconciliation

- Only safe card metadata is stored: card ID, status, last four, scope, and timestamps. Encrypted PAN/CVC fields are discarded immediately.
- Stable 64-character idempotency keys, bounded timeouts/retries, same-origin redirects, response limits, safe error codes, and UUID validation are enforced.
- If the deliberate challenge unexpectedly authorizes, Lazarus requests a reversal and still records a policy violation.
- The public sandbox has no documented cancel endpoint in this flow; Lazarus disables local authority and records expiry_scheduled, and reset is blocked while unexpired authority remains.

SECTION 11

Monad and x402: paid discovery with verifiable settlement

Monad has two roles in the story: an implemented testnet x402 payment for availability intelligence, and a still-local model for bounty, collateral, quorum, and reward release.

Protected-preview payment path

Component	Current behavior
Network	Monad testnet, eip155:10143; official test USDC; default exact price/cap is 10,000 atomic units (0.01 test USDC)
Payer	Dedicated low-value signer. Privy is the managed-wallet design; the current branch can also allow one dedicated raw key only behind protected Preview gates.
Payee	Different receive-only wallet; Lazarus requires only the public address
Seller	Co-located protected x402 availability endpoint with facilitator verification/settlement
Durability	Neon payment-ID/fingerprint/resource table provides exact replay and uncertain-outcome blocking
Evidence	Buyer independently validates the transaction receipt, confirmations, token contract, recipient, amount, and Transfer log

What the chain proves

Proves

A bounded payment authorization settled as the exact expected test-USDC Transfer for the protected availability resource.

Does not prove

That the archive purchase, full bounty registry, verifier network, reward tranches, or reseeding were executed onchain.

Operational rule

An ambiguous result is tombstoned and must be reconciled by stable payment ID before another authorization may be attempted.

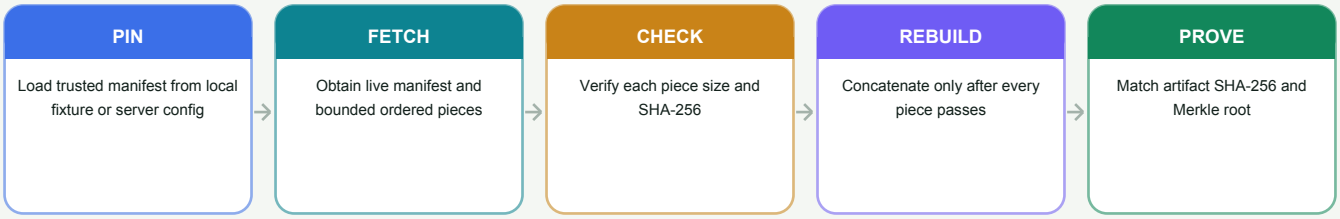
Public production boundary

Public Vercel Production keeps the x402 buyer local and the paid seller disabled. Live testnet execution is restricted to an explicitly armed, deployment-protected branch Preview with isolated state, exact caps, one pinned mission, and disabled reset/arbitrary mission creation.

SECTION 12

Recovery and cryptographic integrity

The commercial result is not accepted merely because a payment succeeded. The artifact must match the sponsor-pinned commitment byte for byte.



Property	Local fixture	Remote provider
Artifact	Bundled CC0 rainfall dataset	Sponsor-pinned artifact configured server-side
Piece count	24	8 in current merchant deployment
Transport	Local file bytes	Authenticated HTTPS, bounded response sizes
Trust root	Generated from the bundled fixture	Complete trusted manifest supplied independently; never learned from provider response
Verification	Piece size/hash, Merkle root, full SHA-256	Exact live-manifest equality plus the same piece/root/artifact checks
Acceptance evidence	Automated exact-byte reconstruction tests	Documented 8/8 acceptance run and root match

Why both SHA-256 and a Merkle root?

Piece hashes

Detect corruption or substitution at the smallest delivered unit and support incremental progress.

Merkle root

Commits to the ordered set of piece descriptors and lets the mission bind recovery to one content structure.

Full artifact hash

Proves that concatenating all verified pieces reconstructs the exact intended file.

Current boundary

Remote retrieval and integrity verification are real. Persistent replica hosting, malware/content scanning, arbitrary signed-manifest ingestion, independent retention proofs, and actual BitTorrent/IPFS reseeding remain future work.

SECTION 13

Policy, security, and failure containment

Lazarus uses layered, fail-closed controls so no single quote, wallet, merchant, card, or serverless retry can silently widen authority.

Gate	Checks	Result
Rights and mission	Rights evidence, active state, deadlines, blocked content, kill switch	Unauthorized or inactive recovery stops before spend
Signed quote	Session, key pin, signature, digest, root, merchant, provider, MCC, purpose, currency, amount, terms, rounds, expiry	Only one binding commercial offer becomes eligible
Payment policy	Rail, recipient, exact amount, per-transaction and aggregate budget, count, approval, duplicate intent	Exact intent is allowed or denied with a stable code
Rain/x402 adapter	Provider-specific destination, caps, identities, redirect policy, response size, retries, idempotency	External calls remain bounded and replay-aware
Recovery proof	Manifest pin, piece hashes, Merkle root, full SHA-256, quorum	Completion and reward milestones fail closed on corruption

Deliberate policy challenge

A meaningful decline, not just an oversized amount

The built-in challenge proposes a $9.00 purchase - inside the numerical ceiling - but uses merchant_luxury_market, MCC 5944, and unrelated_purchase. It is denied because merchant, MCC, and purpose are wrong. A second manual challenge uses $250.00 to demonstrate the amount ceiling.

Server and secret controls

HTTP

Loopback bind locally; Host and same-origin mutation checks; 1 MB JSON limit; restrictive headers; path containment.

Secrets

Environment-only credentials; public state exposes booleans, not values; upstream errors are sanitized.

Card data

Encrypted PAN/CVC never enters state, SSE, audit JSON, browser code, logs, or model prompts.

Replay

Stable idempotency keys; merchant session-to-mission binding; quote consumption; durable x402 payment IDs.

Serverless

Runtime-binding marker, isolated state keys, semantic manifest comparison, and adapter rehydration prevent cross-profile adoption.

Crash safety

External mutations are checkpointed; unknown x402 outcomes block reset/retry until reconciliation.

SECTION 14

Persistence and deployment profiles

Three independent selectors provide flexibility, while Vercel policy constrains unsafe combinations and Neon preserves state across serverless invocations.

Selector	Values	Controls
ADAPTER_MODE	local | rain-sandbox	Local card simulation or authenticated Rain sandbox calls
MONAD_EXECUTION_MODE	local | x402-testnet	Local x402-shaped discovery or capped Monad testnet payment
MERCHANT_MODE	local | remote	Bundled Atlas fixture or authenticated merchant/provider

Deployment profiles

Profile	External behavior	State and safety
Local default	No payment or chain network calls; local merchant and 24-piece fixture	Loopback server; local JSON audit snapshot; reset freely
Remote merchant Production	Authenticated bargain and eight-piece provider; payments/bounty/verifiers/reseeding can remain local	Neon JSONB; isolated merchant-* state namespace; HTTPS, key pin, full trusted manifest
Rain hybrid	Remote Rain sandbox card and transaction calls; USD only	Isolated state; card authority restored after serverless invocation; reset blocked while unexpired
Protected x402 Preview	Capped test-USDC discovery with co-located seller; local fixture only	Deployment Protection; preview-* namespace; one mission; durable payment IDs; reset and new missions disabled

Persistence model

Local

.data/state.json is an audit/debug snapshot written through a temporary file and rename. It is not a general financial ledger.

Vercel

Neon Postgres stores the complete schema-4 snapshot as JSONB with optimistic revision checks.

x402 seller

A separate Neon table stores payment ID, fingerprint, content root, status, result, and settlement for durable replay handling.

Serverless rehydration

Before a mission mutation, Lazarus reloads state and rebuilds adapter ledgers. It restores persisted remote negotiation sessions, Rain authority, and x402 evidence; rejects cross-mission replay; compares manifests semantically despite JSONB key ordering; and downloads only the mutable target's remote pieces.

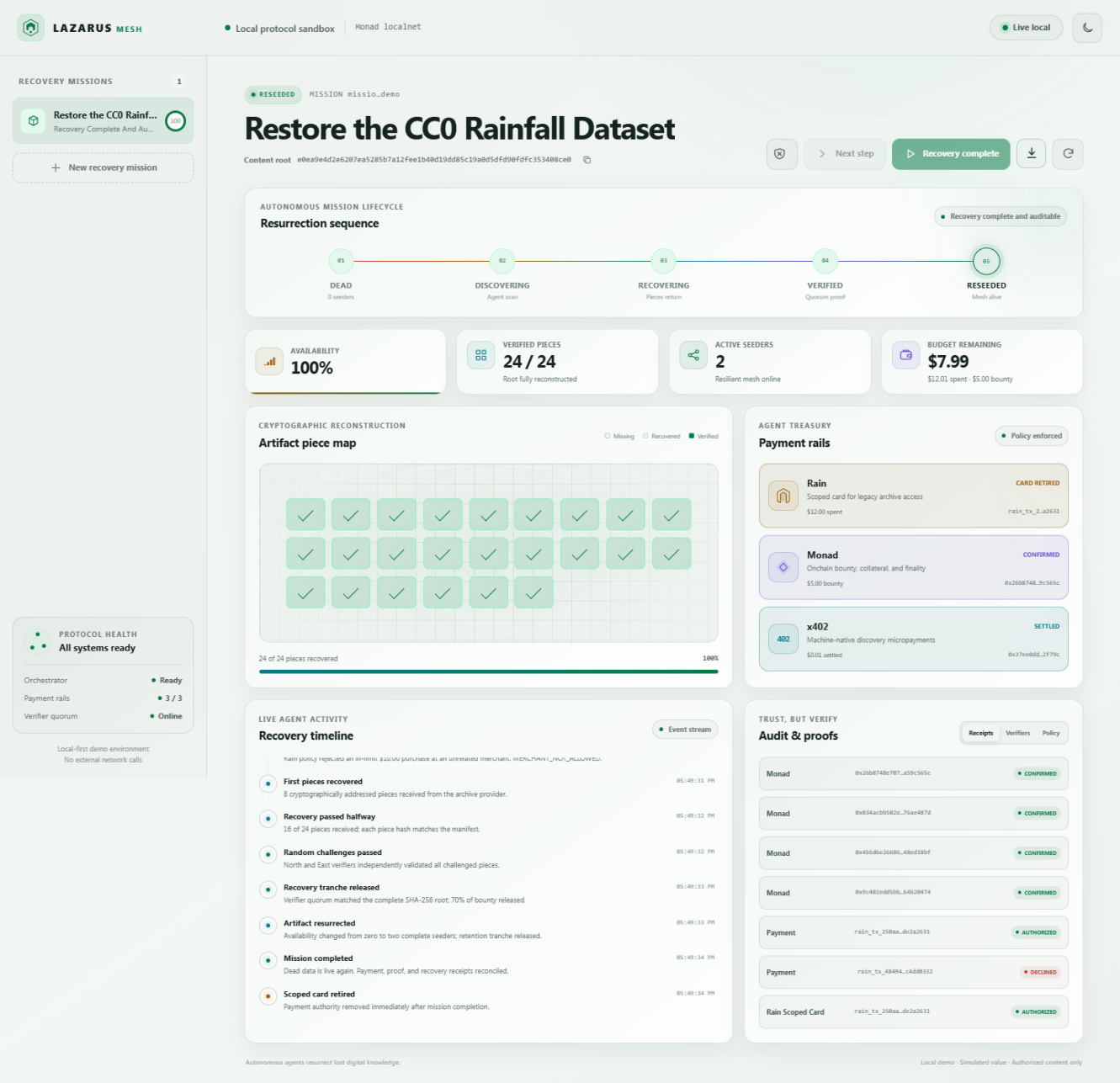
Defense in depth

The serverless wrapper rejects invalid Host/Origin mutations before any external adapter rehydration, and the inner request handler performs the guard again.

SECTION 15

Browser dashboard and operator experience

The dashboard makes autonomy inspectable: lifecycle, pieces, budget, payment rails, bargaining, policy decisions, verifiers, and receipts are visible in one workspace.



Checked-in local-mode desktop QA capture. Values and labels change with the active deployment profile.

Mission control

Create a rights-attested mission, select supported accounting currency, run one step or the full recovery, export audit JSON, and reset where permitted.

Trust and evidence

Deal tab shows ask, counters, quote, savings, terms, signature/session metadata, and policy decision. Receipts and verifier quorum stay separate.

Responsive operation

Semantic HTML, keyboard-accessible tabs, ARIA live regions, theme persistence, SSE locally, and five-second polling on Vercel.

SECTION 16

API surface and developer setup

The HTTP API exposes mission control and public evidence without exposing credentials or raw card/recovery data.

Method	Route	Purpose
GET	/api/health	Runtime truth, Rain health, and optional read-only Monad/x402 readiness
GET	/api/state	Sanitized application state and server-published mission-creation policy
GET	/api/events	Server-sent state updates; browser polls on serverless deployment
GET	/api/x402/availability/:root	Protected paid resource; disabled in public Production
POST	/api/demo/reset	Reset where no unresolved authority exists and profile permits it
POST	/api/missions	Create one runtime-pinned, rights-attested mission
POST	/api/missions/:id/step	Execute the next action
POST	/api/missions/:id/run	Execute all remaining actions
GET/POST	/api/missions/:id/negotiation	Read or idempotently execute bargaining after discovery
POST	/api/missions/:id/blocked-purchase	Exercise a second policy challenge
GET	/api/missions/:id/export	Download formatted audit JSON

Quick start

```
npm install
npm start

# open http://127.0.0.1:4173

# verification
npm test
```

Configuration groups - names only

Integration	Representative environment names
Runtime	ADAPTER_MODE, MONAD_EXECUTION_MODE, MERCHANT_MODE, LAZARUS_STATE_KEY, DATABASE_URL
Rain	RAIN_API_BASE_URL, RAIN_API_KEY, RAIN_USER_ID, RAIN_TEAM_ID, RAIN_CONTRACT_ID, RAIN_AUTO_FUND_MINOR
Merchant/provider	MERCHANT_BASE_URL, MERCHANT_API_TOKEN, MERCHANT_EXPECTED_KEY_ID, MERCHANT_CURRENCY, MERCHANT_TRUSTED_MANIFEST_JSON
Monad/x402	MONAD_RPC_URL, X402_FACILITATOR_URL, X402_AVAILABILITY_BASE_URL, MONAD_PAY_TO_ADDRESS, X402_EXPECTED_AMOUNT_ATOMIC, MONAD_X402_CONFIRMATIONS
Managed signer	PRIVY_APP_ID, PRIVY_APP_SECRET, PRIVY_PAYER_WALLET_ID, PRIVY_PAYER_ADDRESS
Vercel gates	ALLOW_EXTERNAL_WRITES_ON_VERCEL, LIVE_PREVIEW_BRANCH, X402_SELLER_ENABLED

Security rule: real values belong only in ignored local environment files or encrypted deployment controls. Never put keys, wallet secrets, user/team/contract identifiers, PAN/CVC, or private manifests into browser code, prompts, screenshots, Git, or this document.

SECTION 17

Verification and engineering evidence

The repository test suite validates domain boundaries, end-to-end mission behavior, external adapter contracts, serverless replay handling, and protected-preview policy.

186	0	160	8.1s
tests	**failures**	**top-level subtests**	**test duration**
All passing	Loopback-enabled run	Nested cases included	Snapshot run

Area	Representative coverage
Domain and currency	Integer rounding, five-currency isolation, state transitions, stable hashes, rail selection, policy boundary codes
Full mission	Nine-step HTTP run, negotiation, exact economics, rights/budget validation, audit export, reset and concurrency safety
Remote merchant/provider	Authenticated signed quotes, tampering/key substitution, session replay after fresh serverless runtime, manifest/piece corruption and size bounds
Rain sandbox	Endpoint/header/body mapping, idempotency, session encryption, PAN/CVC discard, approved settlement, wrong-MCC decline, reconciliation
Monad/x402	v2 exact requirement, EIP-3009 signer policy, wrong chain/token/payee/price, durable payment ID, receipt Transfer verification, unknown outcomes
Vercel and persistence	Production denial, Preview arming, branch/host/state isolation, JSONB key reordering, pre-rehydration origin guard
UI QA	Checked-in desktop/mobile captures and a browser QA script for rendering, overflow, and audit behavior

What the test result means

High confidence in software behavior, not a substitute for live settlement evidence

The suite uses injected fake network clients for Rain, Monad RPC, facilitator, Privy, and x402 boundaries. It proves validation, sequencing, replay protection, and expected request/response handling without contacting providers or moving tokens. Live Rain evidence comes from the separately documented sandbox run; a live Monad claim requires an actual protected-preview transaction and explorer receipt.

Command executed for this report

```
npm test -> tests 186 | pass 186 | fail 0
```

SECTION 18

Current reality and production roadmap

The MVP is credible because it clearly distinguishes live integration from sandbox/testnet behavior and local economic modeling.

Capability	Status today	Next production step
Remote bargaining	Real authenticated HTTPS; signed and pinned quote	Multi-merchant discovery, cancellation, disputes, SLAs, human escalation
Remote recovery	Real piece transfer and cryptographic verification	Sandboxed workers, malware scanning, arbitrary trusted manifests, durable replicas
Rain	Verified sandbox scoped-card and transaction flow	Production API/compliance review, webhooks, cancellation, disputes, reconciliation
Monad x402	Implemented protected testnet path; public Production disabled	Complete live evidence, managed custody, monitoring, reconciliation, reviewed mainnet policy
Monad bounty	Local deterministic ledger; reference Solidity only	Compile, audit, deploy contract; writer adapter; receipt/reorg/nonce/dispute handling
Verifier quorum	Scripted local roles	Independent identities, signed attestations, challenge and fraud process
Reseeding	Two simulated seeders	Persistent storage nodes, proof of retention, repair/re-replication jobs
Multi-currency	Five deterministic accounting currencies	Approved FX feed, rate locks, fees, treasury, refunds, accounting and tax
Operations	Neon state and durable x402 payment IDs	Normalized ledger/events, queues, webhooks, observability, RBAC, backups, incident response

Release classification

Ready to demonstrate

Remote merchant bargain, signature validation, remote artifact recovery, Rain sandbox controls, local multi-currency workflow, audit dashboard.

Ready only in protected testnet

Capped x402 discovery payment with dedicated payer, distinct payee, protected seller, durable IDs, and exact receipt verification.

Not production-ready

Real-money settlement, deployed bounty registry, independent verification, durable reseeding, production FX, and multi-tenant operations.

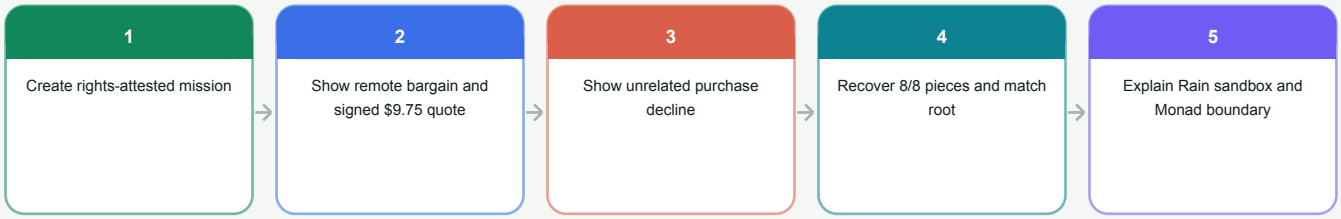
Recommended next milestone

Capture one successful protected-preview x402 transaction with the exact payment ID, explorer link, six confirmations, and verified test-USDC Transfer; then add a durable operator reconciliation screen before expanding to any additional external write.

SECTION 19

Demo narrative, repository map, and conclusion

The strongest presentation follows the mission from zero availability to a verified result, showing the guardrails at the exact moments they matter.



Core repository map

Path	Purpose
server.js / api/index.js	HTTP runtime, same-origin guards, adapter selection, Vercel durable wrapper
src/orchestrator.js	Nine-step workflow, locks, checkpointing, public-state sanitization
src/domain/	Canonical hashing, currency, quote/payment policy, rail choice, state machine
src/services/negotiation-*	Local deterministic or authenticated remote bargaining
src/services/rain-*	Local card model and external Rain sandbox adapter
src/services/x402-* / privy-signer.js	Local handshake, Monad buyer, protected seller, signer, and durable replay
src/services/recovery-*	Local and remote artifact retrieval, piece/root/hash verification
src/neon-state.js / src/rehydrate-local.js	Serverless state persistence and adapter reconstruction
public/	Responsive mission dashboard
contracts/RecoveryBountyRegistry.sol	Unaudited future reference contract; not deployed or invoked
tests/	186 passing domain, integration, adapter, persistence, and policy tests

Live project references

- Buyer demo: lazarus-mesh-merchant-demo.vercel.app
- Merchant service and operator console: lazarus-merchant.vercel.app
- Primary documentation: README.md, PROJECT_GUIDE.md, docs/API_INTEGRATION.md, docs/PRODUCTION_READINESS.md, and docs/HACKATHON_ACCEPTANCE_AND_CURRENCY.md

Final takeaway

Lazarus Mesh is a complete agentic-commerce story: an agent discovers a service, bargains with a real merchant, receives a signed binding quote, creates narrow payment authority, purchases access through a sandbox or testnet-aware rail, verifies the delivered artifact, and retires authority. Its most important innovation is not autonomous spending alone - it is autonomous spending whose commercial terms, payment scope, and delivered result are independently checkable.

Document basis: repository contents and tests at branch preview/x402-live, commit b41f3a6, reviewed 9 August 2026. No credential values or private identifiers are included.